Lab 5 Guide: Getting Started with Cloud Storage and Cloud SQL

About this lab

Most web applications need two kinds of storage: a database for structured data and somewhere to keep files such as images. In this lab you build a small blog-style web page that uses both. A Compute Engine VM runs Apache and PHP, a Cloud SQL instance provides a managed MySQL database, and a Cloud Storage bucket hosts the page's banner image. The same pattern applies to real blogging platforms and other PHP applications.

What you will build

Component	Name	Role
Compute Engine VM	bloghost	Web server running Apache and PHP
Cloud Storage bucket	Your project ID	Holds the public banner image my-excellent-blog.png
Cloud SQL instance (MySQL)	blog-db	Managed database the web page connects to
Database user	blogdbuser	Account the PHP code uses to sign in to blog-db
Web page	index.php	Shows the banner image and the database connection status

Keep a scratch file open. You'll need to note three values along the way: the bloghost external IP, the bloghost internal IP and the blog-db public IP. Region and zone values come from your Lab setup and access panel.

What you will learn

- How to create a Cloud Storage bucket from the command line and share one object publicly
- How to create and configure a Cloud SQL for MySQL instance, including a user and an authorized network
- How to connect a PHP application on a VM to Cloud SQL
- How to display an image from Cloud Storage on a web page

Task 1. Open the lab environment

1. Select **Start Lab**. If the lab requires payment, choose a payment method in the pop-up.
2. Check the **Lab setup and access** panel for your temporary username and password, the **Open Google Cloud console** button, and the region and zone to use. The timer at the top shows how long you have left.
3. Select **Open Google Cloud console**. In Chrome, you can right-click and open it in an Incognito window instead.
4. If asked to pick an account, choose **Use Another Account**.
5. Enter the lab **Username** and select **Next**, then enter the lab **Password** and select **Next**.
6. Accept the terms. Skip recovery options, two-factor authentication and free-trial offers, since this account exists only for the lab.

Important: Use only the lab credentials. Signing in with your own Google Cloud account could lead to charges.

Task 2. Launch the web server VM

You create a VM called bloghost and give it a startup script, so Apache and PHP are installed automatically the first time it boots.

1. Go to **Navigation menu** > **Compute Engine** > **VM instances** and select **Create instance**.
2. Enter the settings below.

Setting	**Value**
Name	bloghost
Region / Zone	From your Lab setup and access panel
Machine type	e2-standard-2
OS and storage	Debian GNU/Linux 12 (bookworm). Select Change if a different image is shown.
Networking > Firewall	Allow HTTP traffic

3. In the left pane, select **Advanced**. Under **Automation**, paste this into the **Startup script** field:

```
#!/bin/bash
apt-get install apache2 php php-mysql -y
service apache2 restart
```

Check the field: the script only runs if it's in the Startup script box. Pasted anywhere else, it does nothing and the web server won't be installed.

4. Leave everything else at the defaults and select **Create**. The VM takes about two minutes to be ready.
5. On the **VM instances** page, copy the internal and external IP addresses of bloghost into your scratch file.

Task 3. Create a bucket and upload the banner image

Bucket names must be unique across all of Google Cloud. Your project ID is already globally unique, so you'll use it as the bucket name. You'll place the bucket in the multi-region (US, EU or ASIA) nearest to your assigned region.

1. Select **Activate Cloud Shell** in the top-right toolbar, and select **Continue** if prompted.
2. Save your chosen location in a variable. Run only the one that matches your region:

If your region is in	Run
North or South America	`export LOCATION=US`
Europe	`export LOCATION=EU`
Asia	`export LOCATION=ASIA`

3. Create the bucket. Cloud Shell already holds your project ID in the DEVSHELL_PROJECT_ID variable. Select **Authorize** if prompted.

```
gcloud storage buckets create -l $LOCATION gs://$DEVSHELL_PROJECT_ID
```

4. Download the banner image from a public training bucket into Cloud Shell:

```
gcloud storage cp gs://cloud-training/gcpfci/my-excellent-blog.png
my-excellent-blog.png
```

5. Upload it to your own bucket:

```
gcloud storage cp my-excellent-blog.png
gs://$DEVSHELL_PROJECT_ID/my-excellent-blog.png
```

6. Make just this one object readable by anyone, by changing its access control list:

```
gsutil acl ch -u allUsers:R
gs://$DEVSHELL_PROJECT_ID/my-excellent-blog.png
```

Task 4. Set up the Cloud SQL database

You create a managed MySQL instance next to the web server, add a database user, and allow only the bloghost VM to connect.

Create the instance

1. Go to **Navigation menu** > **Cloud SQL**, scroll to the bottom of the page and select **Create instance**.
2. Choose **MySQL** as the database engine, then enter the settings below.

Setting	Value
Cloud SQL edition	Enterprise
Edition preset	Sandbox
Instance ID	blog-db
Password	Passw0rd1!

Region	Same region as bloghost
Zonal availability	Single zone
Specify zones > Primary zone	Same zone as bloghost
Show configuration options > Security	Allow unencrypted network traffic (not recommended)

Why these choices? Putting the database in the same zone as the web server gives the best performance. Unencrypted traffic is allowed only because this lab doesn't set up SSL. In production, keep encryption on.

3. Select **Create instance** and wait a few minutes for it to finish deploying.

Add a user and authorize the web server

1. On the blog-db details page, under **Connect to this instance**, copy the **Public IP address** into your scratch file.
2. In the left pane, select **Users** > **Add user account**. Enter user name **blogdbuser** and password **Passw0rd1!**, then select **Add**. Wait for the user to appear.
3. In the left pane, select **Connections** and open the **Networking** tab. If you're asked to choose between private and public IP, choose **Public IP**.
4. Select **Add a network**. If the button is greyed out, the user is probably still being created, so wait a moment.
5. Name the network **web front end**. For **Network**, enter the bloghost **external** IP address followed by /32 (for example, 35.192.208.2/32, but use your own address).
6. Select **Done**, then **Save**. If you see "Another operation is in progress", wait until blog-db shows a green check, then save again.

Common mistake: using the VM's internal IP here. The VM connects to the database's public IP, so Cloud SQL sees the VM's external address. The /32 limits access to that single address.

Task 5. Connect the web page to Cloud SQL

You write a PHP page that tries to connect to the database, see it fail, then fill in the connection details and see it succeed.

Create the page

1. Go to **Compute Engine** > **VM instances** and select **SSH** in the bloghost row. Select **Authorize** if prompted.
2. Move to the web server's document root:

```
cd /var/www/html
```

3. Open a new file called index.php in nano:

```
sudo nano index.php
```

4. Paste in the code below exactly as shown. Leave CLOUDSQLIP and DBPASSWORD as they are for now.

```
<html>
<head><title>Welcome to my excellent blog</title></head>
```

```
<body>
<h1>Welcome to my excellent blog</h1>
<?php
$dbserver = "CLOUDSQLIP";
$dbuser = "blogdbuser";
$dbpassword = "DBPASSWORD";
// In a production blog, we would not store the MySQL
// password in the document root. Instead, we would store
// it in Secret Manager. For more information see
// https://cloud.google.com/sql/docs/postgres/use-secret-manager

try {
 $conn = new PDO("mysql:host=$dbserver;dbname=mysql", $dbuser,
$dbpassword);
  // set the PDO error mode to exception
 $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
  echo "Connected successfully";
} catch(PDOException $e) {
  echo "Database connection failed:: " . $e->getMessage();
}

?>
</body></html>
```

5. Save with **Ctrl+O** then **Enter**, and exit with **Ctrl+X**.
6. Restart Apache:

```
sudo service apache2 restart
```

Check the page before configuring it

1. In a new browser tab, go to the bloghost external IP followed by /index.php (for example, 35.192.208.2/index.php, but use your own address). If the browser warns that the site doesn't support a secure connection, select **Continue to site**.
2. The page shows **Database connection failed: ...**. That's expected, because the code still has placeholder values.

Add the connection details

1. Back in the SSH session, make sure you're still in /var/www/html and open the file again:

```
sudo nano index.php
```

2. Replace **CLOUDSQLIP** with the blog-db public IP address, keeping the quotation marks.
3. Replace **DBPASSWORD** with **Passw0rd1!**, keeping the quotation marks.
4. Save with **Ctrl+O** then **Enter**, exit with **Ctrl+X**, and restart Apache:

```
sudo service apache2 restart
```

5. Reload the browser tab. The page now shows **Connected successfully**.

Note: A real blog wouldn't show the database status to visitors. It's displayed here only so you can confirm the connection works.

Task 6. Add the banner image from Cloud Storage

Finally, you point the web page at the image in your bucket.

Get the image's public URL

1. Go to **Navigation menu** > **Cloud Storage** > **Buckets** and open the bucket named after your project.
2. Find **my-excellent-blog.png**. In its **Public access** column, copy the URL behind the link icon (or behind **Public link**).

If there's no link icon or public link, refresh the page. If it still isn't there, go back to Cloud Shell and check that the gsutil acl ch command in Task 3 ran without errors.

Add the image to the page

1. In the bloghost SSH session, move to the document root and open the file:

```
cd /var/www/html
sudo nano index.php
```

2. Use the arrow keys to put the cursor at the start of the line containing <h1>. Press **Enter** to create a blank line above it, then move up to that blank line.
3. On the blank line, type the image tag around your copied URL, so the line reads like this (with your own URL):

```
<img
src='https://storage.googleapis.com/YOUR_PROJECT_ID/my-excellent-blog
.png'>
```

4. Check that the <img> line now sits directly above the <h1> line, with a single quote before and after the URL.
5. Save with **Ctrl+O** then **Enter**, exit with **Ctrl+X**, and restart Apache:

```
sudo service apache2 restart
```

6. Reload the browser tab. The page now shows the banner image above the heading, followed by **Connected successfully**.

Troubleshooting

What you see	Likely cause	What to do
Browser can't reach bloghost at all	HTTP traffic not allowed, or Apache not installed	Check that Allow HTTP traffic is on and that the startup script was in the Startup script field
Database connection failed, even after editing the file	Wrong IP in the authorized network, wrong password, or Apache not restarted	Confirm the authorized network uses the bloghost external IP with /32, check the values in index.php, and restart Apache

Banner image is a broken icon	Object isn't public, or the URL was typed incorrectly	Re-run the gsutil acl ch command and re-copy the URL from the Storage browser
Add a network button is greyed out	User creation hasn't finished	Wait for blogdbuser to appear, then try again

Review

- You launched a Compute Engine VM that installs Apache and PHP with a startup script.
- You created a Cloud Storage bucket with gcloud, uploaded an image and made that object public.
- You created a Cloud SQL for MySQL instance, added a user and authorized only the web server's external IP.
- You connected a PHP page to Cloud SQL and displayed the image from Cloud Storage.